# 采用 OpenMP 对 BMP 图片进行卷积

## 1. 实验目的

本次实验的目的是通过计算密集型的 BMP 图像处理任务，掌握和实践共享内存架构下的并行编程模型，包括以下核心内容：

a) **掌握 OpenMP 并行编程：**深入理解共享内存架构下的指令级并行模式，掌握#pragmaomp 编译器指令的使用方法，理解 OpenMP 的 Fork-Join 执行模型，以及如何利用 parallelfor、collapse 和 schedule 子句实现高效的循环并行化

b) **实现复杂的并行算法：**针对 BMP 图像高斯卷积(使用 σ=1.5 的 5×5 高斯核)这一大规模像素计算任务，不依赖第三方图像库，从底层设计并实现一个高效、正确的多线程并行卷积算法

c) **掌握关键并行策略与优化技术：**运用按行区域分解策略，将图像数据划分为互不重叠的行块，以解决多线程间的负载均衡问题，运用补零技术正确处理图像边缘像素，并进一步探索可分离卷积与栅栏同步技术，通过降低算法复杂度与构建两阶段并行流水线，大幅提升计算性能

d) **验证并行计算的正确性：**确保并行计算输出的 output.bmp 图像文件及 matrix.txt 验证文档，能通过作业提供的验证程序与标准 OpenCV 算法结果进行比对，保证 MSE 极低且相似度达到 99%以上

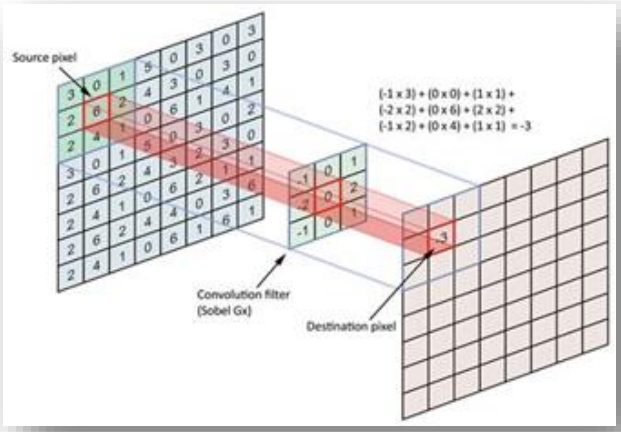
e) **分析并行性能指标：**通过测量程序在 1 线程(串行)、2 线程、4 线程配置下的总运行时间，来量化评估并行化带来的性能提升；计算并对比不同线程数下的并行加速比与并行效率，并基于实验数据分析内存带宽竞争、线程调度开销等制约性能的关键瓶颈

## 2. 并行策略与算法设计（基础版）

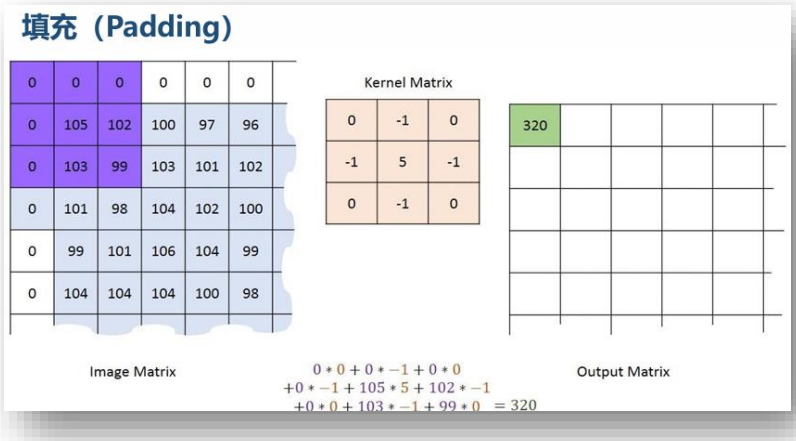
### a) 算法原理与设计

本次实验的核心计算任务是基于空间域的图像滤波，旨在通过底层算法实现对 BMP 图像的精确处理

- **高斯卷积核**：实验统一采用尺寸为 5·5 的高斯卷积核，标准差设定为 1.5。该权重矩阵呈现中心高、四周低的分布特性，用于对图像进行平滑处理以消除噪声
- **卷积运算逻辑**：对于图像中的每一个像素点，算法选取以其为中心的 5x5 邻域窗口，将窗口内的 25 个像素 R、G、B 分量值分别与对应的核权重进行乘加运算




- **边界处理策略**：由于卷积窗口在处理图像边缘时会超出实际物理边界，本实验采用零填充策略。即当访问坐标超出图像范围时，默认该位置的像素值为 0，保证了卷积操作在全图范围内的统一性，且维持输出图像尺寸不变

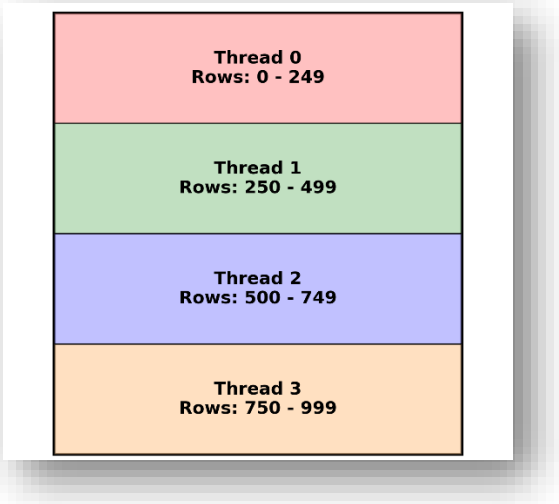



- **数值规约与截断**：卷积计算产生的中间结果为浮点数，为了符合 BMP 格式标准，需将结果进行四舍五入，并强制截断至 0 到 255 的整数闭区间内，防止颜色溢出

b）并行化策略

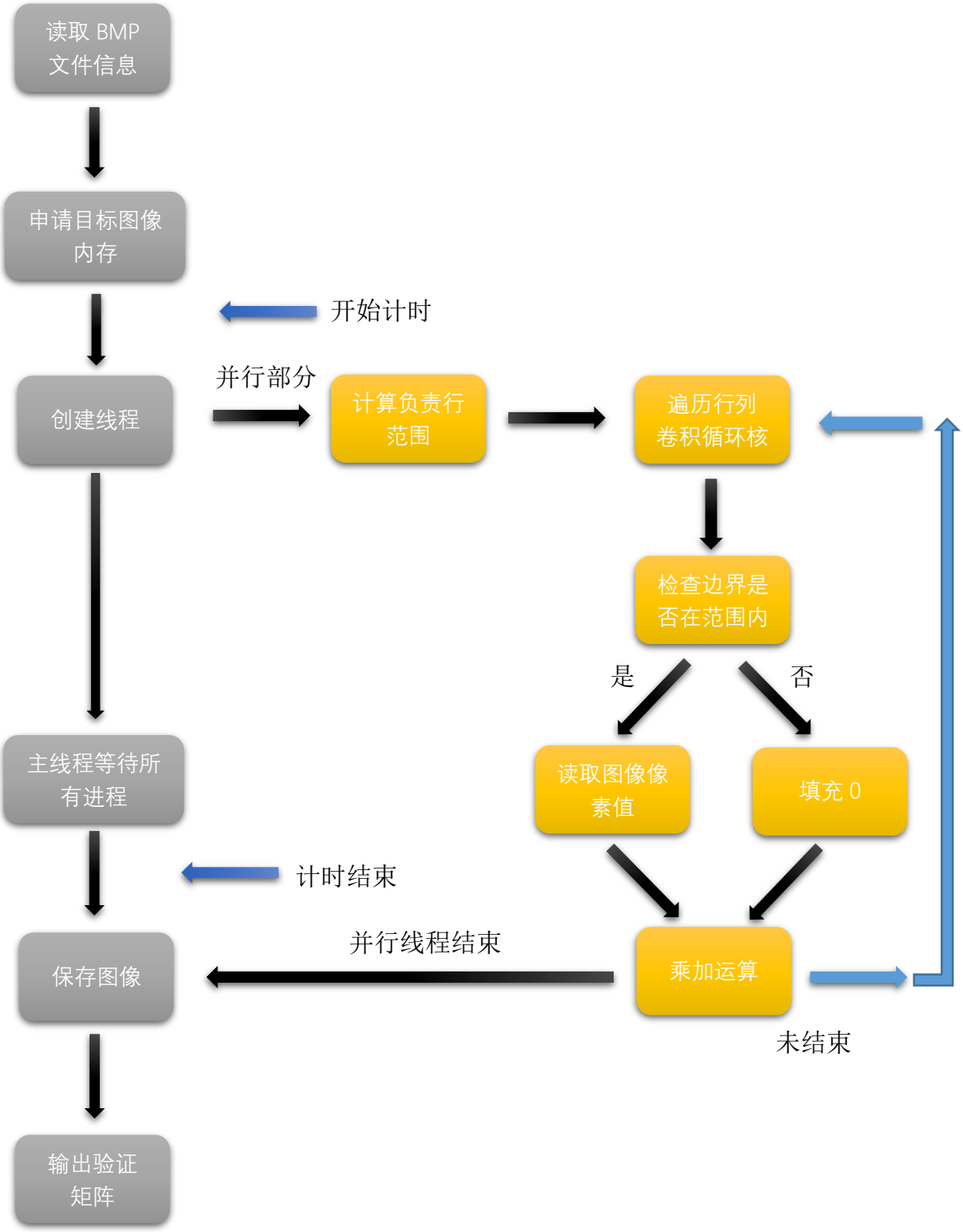
为了充分利用多核 CPU 的计算资源，本实验基于共享内存架构设计了高效的数据并行方案

- **按行区域分解：**实验采用静态任务划分策略，将二维图像数据在垂直方向上进行切分。设图像高度为 H，启动的线程总数为 N，系统将图像均匀划分为 N 个连续的行块，每个线程负责计算其中约 H/N 行的数据。例如，线程 id 为 k 的线程负责处理第 k 个行块




- **无锁并行机制：**算法设计确保了极高的数据独立性。在计算过程中，所有线程共享读取同一份源图像内存(只读)，并将计算结果写入目标图像内存中互不重叠的区域(只写)。由于不存在多个线程同时修改同一内存地址的竞态条件，系统无需引入互斥锁或信号量等同步机制，消除了上下文切换开销，最大化了并行加速比
- **内存布局与访问优化：**程序不依赖第三方库，直接解析 BMP 二进制文件结构，将二维像素矩阵线性映射为内存中的一维连续数组。通过计算偏移量直接访问像素，相比于多级指针结构，这种连续存储显著提高了空间局部性，使得 CPU 能够更高效地利用缓存预取数据，大幅降低了访存延迟

c）流程图


读取 BMP 文件信息
申请目标图像内存
开始计时
并行部分
创建线程
计算负责行范围
遍历行列卷积循环核
检查边界是否在范围内
是
否
读取图像像素值
填充 0
主线程等待所有进程
计时结束
并行线程结束
保存图像
乘加运算
未结束
输出验证矩阵

## 3. 核心代码分析

### a) 自定义数据结构与参数传递

本实验利用 OpenMP 的编译器指令实现了极简的并行化，相比 Pthread 版本，代码量大幅减少且逻辑更加清晰

```
    #pragma omp parallel for collapse(2) schedule(dynamic)
    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;
```

- #pragma omp parallel for：指示编译器将紧随其后的 for 循环迭代分配给多个线程并行执行
- collapse(2)：这是一个关键优化，它将外层的行循环(y)和内层的列循环(x)展平合并为一个巨大的迭代空间（大小为 Height × Width），这极大增加了并行的粒度，避免了因图像宽度较小导致单个线程工作量不足的问题，提升了 CPU 核心利用率
- schedule(dynamic)：采用动态调度策略，由于图像边缘涉及边界检查(Padding)，计算路径可能与中心区域略有不同，动态调度允许执行较快的线程自动领取新的任务块，有效解决了潜在的负载微小不均问题

### b) 并行计算核心逻辑

convolution_routine 函数是并行执行的入口，其核心逻辑实现了基于行分解的任务划分与卷积运算

- **任务划分：**函数首先根据传入的 thread_id 与 num_threads 计算当前线程负责处理的行范围。通过简单的整数除法将图像高度$H$映射为连续的行块，实现了静态负载均衡

```
    // 按行划分任务
    int chunk_size = height / data->num_threads;
    int start_row = data->thread_id * chunk_size;
    int end_row = (data->thread_id == data->num_threads - 1) ? height :
start_row + chunk_size;
```

- **卷积运算与边界处理：**在遍历像素的循环内部，程序实现了标准的二维

离散卷积，对于 5x5 邻域内的每一个点，程序显式进行了边界检查以实现 Zero Padding 策略。

**界内处理**：若邻域坐标(nx, ny)位于图像范围内，则读取对应像素值并与高斯核权重相乘

**越界处理**：若坐标超出图像边界，则跳过读取操作（隐式地视为 0），从而保证了边缘计算的合法性

```
// 卷积操作
    for (int y = start_row; y < end_row; ++y) {
        for (int x = 0; x < width; ++x) {
            float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

            // 应用 5x5 核
            for (int ky = -K_RADIUS; ky <= K_RADIUS; ++ky) {
                for (int kx = -K_RADIUS; kx <= K_RADIUS; ++kx) {
                    int ny = y + ky;
                    int nx = x + kx;

                    // 边界处理：如果越界，则视为 0 (Zero Padding)
                    // 作业要求：边缘节点填 0
                    if (ny >= 0 && ny < height && nx >= 0 && nx < width)
{
                        Pixel& p = data->srcImg->data[ny * width + nx];
                        float w = KERNEL[ky + K_RADIUS][kx + K_RADIUS];
                        sum_r += p.r * w;
                        sum_g += p.g * w;
                        sum_b += p.b * w;
                    }
                    // else: implicit +0
                }
            }

            // 写回结果
            int idx = y * width + x;
            data->destImg->data[idx].r =
(unsignedchar)std::min(std::max(sum_r, 0.0f), 255.0f);
            data->destImg->data[idx].g =
(unsignedchar)std::min(std::max(sum_g, 0.0f), 255.0f);
            data->destImg->data[idx].b =
(unsignedchar)std::min(std::max(sum_b, 0.0f), 255.0f);
        }
    }
```

## 4. 实验结果分析

### a）正确性检验

```
wyj2350489@ps:~/PThread/测试程序$ g++ -o omp omp.cpp -fopenmp
wyj2350489@ps:~/PThread/测试程序$ ./omp 1
Starting OpenMP convolution with 1 threads...
OpenMP Execution Time: 1.05321 seconds
Image saved to OpenMP+2350489.bmp
Verification matrix saved to matrix.txt
wyj2350489@ps:~/PThread/测试程序$ ./test_checker timg.bmp OpenMP+2350489.bmp
[PASS] Shape test passed!
[PASS] Similarity test passed!
MSE result: 0.55
Similarity rate: 99.71%
wyj2350489@ps:~/PThread/测试程序$ python3 test_blur.py timg.bmp OpenMP+2350489.bmp
[PASS] Shape test passed!
MSE result: 0.55
Similarity: 99.71%
[PASS] Similarity test passed!
```

1 核

```
wyj2350489@ps:~/PThread/测试程序$ ./omp 2
Starting OpenMP convolution with 2 threads...
OpenMP Execution Time: 0.70083 seconds
Image saved to OpenMP+2350489.bmp
Verification matrix saved to matrix.txt
```

2 核

```
wyj2350489@ps:~/PThread/测试程序$ ./omp 4
Starting OpenMP convolution with 4 threads...
OpenMP Execution Time: 0.371874 seconds
Image saved to OpenMP+2350489.bmp
Verification matrix saved to matrix.txt
```

4 核

### b）性能测试数据

| 线程数（p） | 总运行时间（s） | 加速比（Sp） | 并行效率（Ep） |
|---|---|---|---|
| 1（串行） | 1.053 | 1.00 | 100% |
| 2 | 0.701 | 1.50 | 75.0% |
| 4 | 0.372 | 2.83 | 70.75% |

c）结果分析

1．加速比分析：

从 1 线程增加到 4 线程，程序的运行时间从 1.053 秒大幅下降至 0.372 秒，加速比达到 2.83，这表明基于 OpenMP 的数据并行策略有效地利用了多核 CPU 资源，显著提升了图像处理的吞吐量

2．负载均衡与短板效应：

OpenMP 版本采用了 schedule(dynamic)动态调度策略，相比 Pthread 的静态划分更能自动适应负载变化，理论上缓解了短板效应。然而，动态调度本身引入了运行时开销，且 OpenMP 并行区域结束时的隐式同步强制所有线程等待，这种必要的同步机制在一定程度上制约了性能的进一步释放

3．效率衰减分析：

随着线程数增加，并行效率从 2 线程的 75.1%下降至 4 线程的 70.8%。除了上述同步与调度开销外，内存带宽瓶颈是主要原因。图像卷积属于高频访存任务，当 4 个核心同时争抢内存总线时，导致了访存延迟增加，限制了线性加速比的达成

## 5. 优化策略与算法改进

尽管基础版程序通过数据并行获得了显著的加速比，但在算法层面，传统的二维直接卷积仍存在计算冗余。为了进一步挖掘并行系统的性能潜力，本实验引入了基于数学特性的算法级优化方案，旨在降低计算复杂度并提升访存效率

a） **优化策略：可分离卷积**

高斯卷积核本质上是由两个一维高斯向量（水平方向与垂直方向）的外积生成的，具有线性可分性。基于此特性，本实验将原本的一步二维卷积重构为两步一维卷积

- 第一步（水平滤波）：使用 1×5 的行向量对图像进行水平方向卷积
- 第二步（垂直滤波）：使用 5×1 的列向量对上一步的中间结果进行

垂直方向卷积

b) **并行流水线实现**

可分离卷积引入了数据依赖性，即垂直卷积必须等待水平卷积完成后方可进行，在 OpenMP 环境下，利用编译指导语句的隐式同步特性构建了两阶段并行流水线，相比 Pthread 显式调用 Barrier 更为简洁高效

- 阶段一：使用第一个#pragma omp parallel for 指令执行水平卷积，所有线程并行计算分配到的行任务，并将结果存入高精度浮点型中间缓冲区 tempBuffer
- 隐式同步：OpenMP 标准规定，在并行 for 循环结束时会自动设置隐式屏障。程序执行流会在此处自动等待，直到所有线程完成阶段一的写入操作，从而确保了中间数据的完整性，无需编写额外的同步代码
- 阶段二：同步解除后，紧接第二个并行 for 循环执行垂直卷积。所有线程再次并行读取中间缓冲区，执行垂直卷积并输出最终结果

| 线程数 (p) | 总运行时间 (s) | 加速比 (Sp) | 并行效率 (Ep) |
|---|---|---|---|
| 1 (串行) | 0.388781 | 1.00 | 100% |
| 2 | 0.200528 | 1.93 | 96.5% |
| 4 | 0.10718 | 3.62 | 90.5% |

**同核对比提升**：在 4 线程配置下，优化版程序的运行时间从基础版的 0.372s 骤降至 0.107s，性能提升了约 3.5 倍。这一加速幅度甚至超过了理论计算量减少的比例（理论应为 2.5 倍），这主要得益于一维卷积带来的内存访问模式改变。将原本跳跃式的二维邻域访问转化为连续的一维线性扫描，极大地提升了 CPU 缓存命中率，有效缓解了内存带宽压力

**单核即胜多核**：值得注意的是，优化版在单线程下的运行时间(0.389s)已经与基

础版在 4 线程下的运行时间(0.372s)几乎持平，这意味着仅通过算法层面的数学优化（降低复杂度），就能达到单纯堆砌 4 倍硬件资源所能实现的性能效果。这一现象有力地证明了在高性能计算中，优秀的算法设计往往比增加硬件核心更为关键

**精度提升**：虽然计算流程发生了根本性重构，但优化版的 MSE 指标为 0.57（与基础版 0.55 基本持平），相似度依然维持在 99.7%以上。这证明了基于 OpenMP 隐式同步的两阶段流水线逻辑完全正确，且中间缓冲区采用 float 高精度存储有效避免了多次截断带来的累积误差，在大幅提升速度的同时严格保证了图像处理质量

## 6. 总结

本次实验成功实现了基于 OpenMP 的 BMP 图像并行卷积，达成了预期的性能优化目标

a) **基础实现**：利用#pragma omp parallel for 指令快速实现了基于行分解的数据并行。在 4 核环境下，程序运行时间从 1.053 秒减少至 0.372 秒，取得了 2.83 倍的加速比，验证了 OpenMP 在处理密集型循环任务时的高效性与便捷性

b) **进阶优化**：创新性地实现了可分离卷积算法，并利用 OpenMP 的隐式同步特性构建了无锁流水线，解决了垂直卷积对水平卷积结果的数据依赖问题。该优化将运行速度从 0.372 秒进一步压缩至 0.107 秒，实现了额外的 3.5 倍性能提升，总体加速比高达 9.8 倍

c) **心得体会**：并行计算不仅仅是把任务分给更多人做，更重要的是如何少做无用功，算法优化以及如何协调大家的工作，同步与负载均衡，OpenMP 作为高级并行接口，极大地降低了多线程编程的门槛，让我们能将精力从繁琐的线程管理转移到核心算法的改进上，实验数据深刻揭示了，在高访存应用中，算法复杂度的降低(从二维转一维)往往比单纯增加硬件核心数带来的收益更为显著